

MiniTLS Writeup

Introduction

MiniTLS is a lightweight TLS implementation built on top of our existing TCP layer. Our overall goal was to establish reasonably meaningful cryptographic safeguards for our TCP stack to prevent attacks like eavesdropping, tampering, impersonation, and man-in-the-middle (MitM). While the official TLS protocol (RFC 8446) is far too extensive to implement in full, our approach focuses on two meaningful components: an authenticated key exchange handshake and an encrypted record layer for sending and receiving data.

In terms of execution, we achieved our target goal. We built a working authenticated Diffie-Hellman Key Exchange (DHKE) using [X25519](#) for the key exchange and [Ed25519](#) for signatures, [HKDF-SHA256](#) for key derivation, and [AES-256-GCM](#) for encrypting and authenticating all post-handshake records. The result is a TLS-like API sitting above TCP that offers confidentiality, integrity, and mutual authentication per connection.

Design / Implementation

The implementation lives in the [tlsStack](#) package and is organized across 5 files: [types.go](#) (data types and message definitions), [handshake.go](#) (handshake logic), [crypto.go](#) (all cryptographic primitives), [doTls.go](#) (TCP interactions), [tlsApi.go](#) (public API used by REPL), and [tlsRepl.go](#) (self explanatory). The public API is pretty similar to TCP:

```
Go
VTLS Dial(addr, port, config) (*VTLSConn, error)
VTLS Listen(port, config) (*VTLSListener, error)
VTLS Accept() (*VTLSConn, error)
VTLS Write(data []byte) (int, error)
VTLS Read(buf []byte) (int, error)
VTLS Close() error
```

Handshake

The handshake is a three-message authenticated DHKE. Both sides already know the other side's [Ed25519](#) public key before starting the handshake. First, the client generates an ephemeral [X25519](#) keypair and sends its public DH value to the server. The server generates its own ephemeral [X25519](#) keypair, signs the concatenation of its DH public value and the

client's DH public value using its **Ed25519** signing key, and sends both DH values plus the signature back. The client verifies this signature (or aborts if verification fails), concatenates and signs both public values, and sends it back to the server to verify. Both sides independently compute the **X25519** shared secret and run **HKDF-SHA256** over it to compute two directional **AES-256-GCM** keys: one for client-to-server traffic and one for server-to-client traffic.

VTLSConn/Listener

```
Go
type VTLSConn struct {
    writeKey, readKey []byte           // directional AES-256-GCM keys
    writeSeq, readSeq uint64           // per-direction nonce counters
    conn               *VTCPConn    // underlying TCP connection
    readBuf            []byte           // leftover plaintext from partial reads
}
```

VTLSConn is the per-connection handle returned after a completed handshake. It stores important keys for encrypted communication and wraps a **VTCPConn**.

```
Go
type VTLSListener struct {
    listener      *VTCPListener // underlying TCP listener
    signingKey    ed25519.PrivateKey
    clientVerifyKey ed25519.PublicKey
}
```

VTLSListener wraps a TCP listener and holds the server's long-term signing key and the trusted client verify key for handshake authentication.

Sending/Receiving Records

Post-handshake, all data travels in encrypted chunks, or “records.” Each call to **VTLSWrite** encrypts the plaintext using **AES-256-GCM** with the **conn**’s write key. The 12-byte nonce is retrieved by encoding the **writeSeq** counter stored in the **TLSCConn**. **AES-GCM**’s **Seal** produces a ciphertext with a 16-byte authentication tag appended and a length prefix is added to the record before sending it via **TCP**.

VTLSRead first checks an internal **readBuf** for any leftover plaintext from a previous partial read and returns that immediately if available. Otherwise it blocks waiting for the next record: it reads the 4-byte length prefix, reads exactly that many bytes of ciphertext+tag, and calls

AES-GCM's `Open` to verify the authentication tag and decrypt. Any plaintext that didn't fit in the caller's buffer is saved to `readBuf` for the next call.

REPL

We added a set of TLS-specific REPL commands that parallel our existing TCP commands: `tlsa` (like `tcp 'a'`), `tlsc`, `tlss`, `tlsr`, `tlsc`, and `tls/s` (list open TLS connections). Open TLS connections are tracked in a map in `VTLSStack` from integer connection IDs to `VTLSConn` pointers. This map is used for listing TLS specific connections tracking/removing them.

Discussion / Results

Everything described here works as intended. We were able to run `tlsa` and `tlsc` to complete the authenticated handshake, then use `tlss` and `tlsr` to exchange encrypted messages, and finally close TLS connections with `tlsc`. We also confirmed that both sides derive matching keys.

One non-trivial design challenge was the semantics of `VTLSRead`. Our TCP `VRead` returns up to `len(buf)` bytes of whatever is currently in the receive buffer, blocking only when the buffer is completely empty. Replicating this at the TLS layer is harder because TLS records are discrete sealed units: you cannot partially decrypt a record, and you cannot know whether another record is immediately available in the TCP buffer without attempting a read that might block. Our final implementation returns leftover plaintext from `readBuf` immediately without touching TCP or a single decrypted record. This means a call to `VTLSRead` may return fewer bytes than requested even if multiple records are sitting in the `recvBuf` unlike `VRead` which would return all available bytes. We realized that this situation is probably unavoidable with TLS, and even the `readBuf` we created to make the REPL run smoother could pose a potential security risk in some real-world uses since it stores unencrypted plaintext in a buffer somewhere.

AI Use

We used Claude throughout this project, consistent with what we described in our proposal. The AI was useful for understanding all of the cryptographic methods we were using -- for example, explaining why the server must echo the client's DH public value in its signed message, why nonces must be unique per key, and what AES-GCM's Seal and Open actually do under the hood. It was also helpful for choosing which library packages to use (`crypto/ecdh`, `crypto/ed25519`, `crypto/cipher`, `golang.org/x/crypto/hkdf`) and explaining their interfaces.

We made all of our own design decisions, especially as they related to our already-implemented TCP. There were some cases when Claude failed to understand our TCP stack thoroughly enough to write useful code. One such case was in reconciling the difference between `VRead` and `VTLSRead` as described above. It also really liked to overcomplicate our data structures and add wrapper structs in unnecessary places.

Conclusions / Future Work

MiniTLS gave us a much more concrete understanding of how TLS works and why each piece is necessary. The handshake proved to be far more complicated than we initially imagined, but implementing it helped us understand how cryptography and TCP work together to provide network security. It was fun to continue to build on top of our TCP implementation to send/receive encrypted messages.

If we were to continue, implementing session resumption, or persistent local handshake state to save reconnecting clients from doing key exchange again, would be really cool. That was one of our original stretch goals. `TLSSendFile` and `TLSRecvFile` would also probably be straightforward to add on top of the existing `VTLSWrite` and `VTLSRead` interface. Additionally, adding a certificate layer would make the authentication model more robust, since the current pre-shared key assumption is not realistic.